Recurrent networks on SmartWatch gestures

Vincent Brand | 1447806 | Machine Learning – MADS, HAN
2026 | https://github.com/vincentbrand/MADS-ML-Vincent

1 Hypothesis

The gestures are short, variable-length sequences of 3-axis acceleration, and a gesture is essentially a sequence of *local* motion primitives (a flick, a turn) rather than one long-range dependency. I therefore hypothesise that (a) a 1D-convolutional front-end before the recurrent layer will help, because Conv1D extracts these local temporal features that the GRU then orders in time, and (b) GRU and LSTM will perform almost equally, because the sequences are short enough that the LSTM's extra cell-state gate buys little over the GRU's lighter update/reset gates. Random guessing is 5% (1/20 classes), so any working model should far exceed that.

2 Experiment design

The baseline is the provided GRU (`hidden_size`=64, `num_layers`=1), trained with `CrossEntropyLoss`, Adam and `ReduceLROnPlateau` for up to 100 epochs with early stopping (patience 10). From there I varied hidden size (64/128/256), depth (1/2/3 layers), dropout ($0 \rightarrow 0.3$) and the architecture family: plain GRU, LSTM, and a Conv1D+GRU hybrid (which `permute`s $(B, T, F) \rightarrow (B, F, T)$ for the convolution and back before the GRU). Every run is logged to MLflow under a descriptive `model` tag, and Fig. 1 compares their final validation accuracy.

3 Results

All six architectures comfortably passed the 90% target (Fig. 1): even the GRU baseline reached **97.97%**, roughly 20× the 5% random floor. The best model is the **Conv1D+GRU** (conv_filters=128, kernel=5, h=256, 2 layers) at **99.53%** (test loss 0.033), ahead of the best LSTM (98.91%) and the best plain GRU (98.59%). Adding the convolutional front-end gave the single largest jump over a same-sized GRU, while GRU and LSTM landed within ≈ 0.3 percentage points of each other. Scaling the GRU from h=64, 1 layer to h=128, 2 layers helped (97.97%→98.59%) but with clearly diminishing returns.

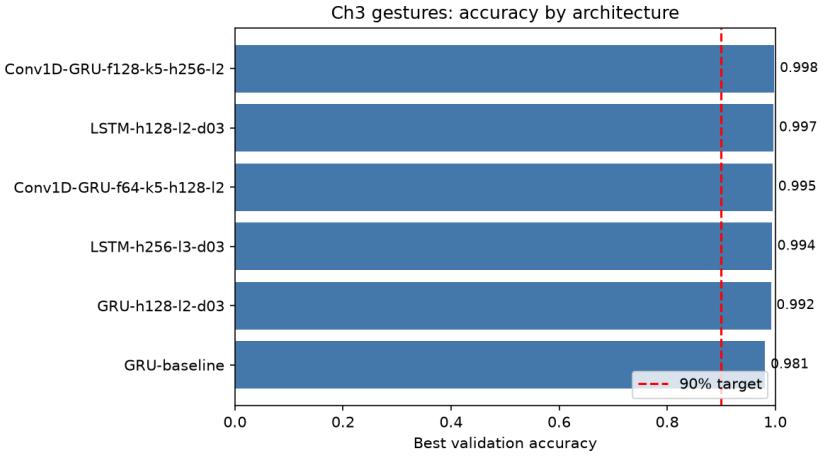

Figure 1: Final validation accuracy per architecture on the gestures dataset, coloured by family (GRU / LSTM / Conv1D+GRU). All models clear the 90% target; the Conv1D+GRU hybrid is best.

4 Conclusions

Both hypotheses held. (a) The Conv1D+GRU won, which fits the data: accelerometer gestures are built from local motion shapes, and convolution captures those cheaply before the GRU models their order. (b) GRU ≈ LSTM here, so the LSTM's extra cell state did not pay off on these short sequences and the cheaper GRU is the better default. The main caveat is that the task is easy — even the baseline hit 98%, so differences between architectures are small; a harder cross-user split or a tighter

epoch budget would separate them better. One thing that did *not* work as expected: dropout is a no-op with `num_layers`=1, since PyTorch only applies it *between* stacked RNN layers.